TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Navegación con Expo Router

Unidad 3 · Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

UNIDAD 3 · NAVEGACIÓN

El problema

Tenés una pantalla. Andás bien.

Ahora necesitás **otra**. Y volver. Y que el botón «atrás» del teléfono haga lo correcto. Y que si alguien abre un enlace a un producto, la app arranque ahí.

En la web eso lo resuelven la URL y el navegador web. En una app **no hay navegador web**: hay que construir esa parte.



¿Y ahora cómo llego al detalle?

Dos formas de resolverlo

REACT NAVIGATION · REGISTRÁS CADA PANTALLA

```
const Stack = createNativeStackNavigator();

<NavigationContainer>
  <Stack.Navigator>
    <Stack.Screen name="Inicio"  component={Inicio} />
    <Stack.Screen name="Detalle" component={Detalle} />
  </Stack.Navigator>
</NavigationContainer>

navigation.navigate('Detalle', { id: 42 });
```

EXPO ROUTER · LA CARPETA ES LA CONFIGURACIÓN

src/app/

- **_layout.tsx** declara el <Stack />
- index.tsx → /
- detalle.tsx → /detalle

```
<Link href="/detalle?id=42">Ver detalle</Link>
```

Creás un archivo y ya existe la ruta. No hay lugar central donde registrar nada.

No compiten: Expo Router está construido encima de React Navigation. Usamos Expo Router —viene en la plantilla, trae enlaces profundos (deep links) y web sin trabajo extra— y como por debajo es React Navigation, su documentación te sirve igual.

UNIDAD 3 · NAVEGACIÓN

La carpeta es el mapa

Antes de ver los navegadores, la regla que ordena todo lo demás: en Expo Router **cada archivo de `src/app/` es una ruta**. No hay ningún lugar donde registrarlas.

```
src/app/
├── _layout.tsx  envuelve todo
├── index.tsx  → /
├── (tabs)/  grupo: no sale en la URL
│   └── perfil.tsx  → /perfil
├── producto/
│   └── [id].tsx  → /producto/42
├── docs/
│   └── [...ruta].tsx  → /docs/a/b/c
└── +not-found.tsx  cualquier ruta inválida
```

- `_layout.tsx` — el navegador (router) de esa carpeta.
- `[id]` — un parámetro.
- `[...ruta]` — captura el resto completo.
- `(grupo)` — organiza sin tocar la URL.
- `+not-found` — la ruta que no existe.

> Como cada pantalla tiene URL real, los deep links y la versión web salen sin configurar nada.

UNIDAD 3 · NAVEGACIÓN

Tres formas de moverse



Stack Tabs Drawer

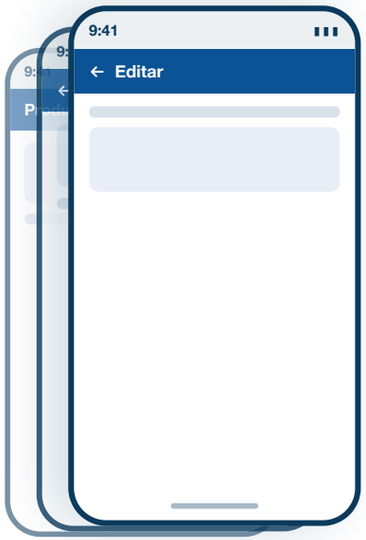
Stack: una pantalla sobre otra



Empezás con una pantalla: la lista.

UNIDAD 3 · NAVEGACIÓN

Stack: una pantalla sobre otra



Cada pantalla nueva **tapa** a la anterior — no la destruye. Por eso «atrás» siempre sabe adónde volver.

Stack: el código

La **carpeta** declara qué pantallas existen; el `_layout.tsx` declara **cómo se ven**.

LA CARPETA

src/app/

- **_layout.tsx** el <Stack />
- index.tsx → /
- detalle.tsx → /detalle
- carrito.tsx → /carrito

`detalle.tsx` no aparece en el layout y funciona igual: hereda las `screenOptions`. Sólo se nombra lo que necesita algo distinto.

SRC/APP/_LAYOUT.TSX

```tsx
import { Stack } from 'expo-router';

export default function Layout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: '#0B3A5D' },
        headerTintColor: '#fff',
      }}
    >
      <Stack.Screen name="index"
        options={{ title: 'Productos' }} />
      <Stack.Screen name="carrito"
        options={{ presentation: 'modal' }} />
    </Stack>
  );
}
```

`screenOptions`
Se aplica a **todas** las pantallas del Stack.

`options`
Sólo a esa pantalla. Pisa lo anterior.

`presentation`
`'modal'` entra desde abajo: carrito, filtro, formulario corto.

`headerShown`
En `false`, sin header. El botón de volver aparece solo.

El header

No es parte de tu pantalla: lo dibuja el navegador (router). Vos le pasás opciones.




```
// El título depende de datos que todavía no llegaron
export default function Producto() {
  const { data } = useProducto(id);
  return (
    <>
      <Stack.Screen options={{ title: data?.nombre ?? 'Cargando…' }} />
      <Vista producto={data} />
    </>
  );
}
```

Opciones del header

Todo lo que se ve en esa barra sale de `options`: el texto, los colores y los botones. Los botones son componentes tuyos, así que pueden tener ícono, `onPress` y etiqueta de accesibilidad.

UN HEADER CON ACCIÓN

```
import { Stack } from 'expo-router';
import { Pressable } from 'react-native';
import Ionicons from '@react-native-vector-icons/ionicons';

<Stack.Screen options={{
  title: 'Zapatillas running',
  headerStyle: { backgroundColor: '#0B3A5D' },
  headerTintColor: '#fff',
  headerTitleStyle: { fontWeight: '700' },
  headerRight: ({ tintColor }) => (
    <Pressable onPress={compartir} hitSlop={12}
               accessibilityLabel="Compartir">
      <Ionicons name="share-outline"
                size={22} color={tintColor} />
    </Pressable>
  ),
}} />
```

EL REPERTORIO

texto	`title` · `headerTitle`, que también acepta un componente — un logo, por ejemplo
colores	`headerStyle` el fondo · `headerTintColor` la flecha y el título · `headerTitleStyle` la tipografía
acciones	`headerRight` y `headerLeft`: devuelven un componente y reciben `tintColor`
forma	`headerTransparent` flota sobre el contenido · `headerShadowVisible` saca la línea de abajo
buscador	`headerSearchBarOptions` pone el buscador nativo del sistema

> `headerRight` recibe `tintColor`: si se lo pasás al ícono, el botón acompaña solo el color del header y no hay que repetir el valor en dos lugares.

UNIDAD 3 · NAVEGACIÓN

Apagar el header

El header lo pone el navegador (router), así que también se saca desde ahí: `headerShown: false`. Hay tres situaciones en las que estorba.

Pantalla completa

Una portada, un onboarding, un login, un mapa, una galería de fotos: el contenido tiene que llegar hasta el borde.

Header propio

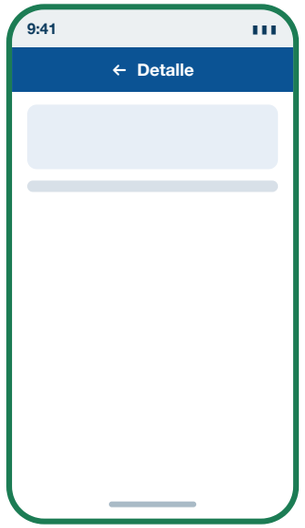
El diseño pide un buscador, un avatar o un degradé. Lo que el navegador no sabe dibujar, lo dibujás vos.

Header doble

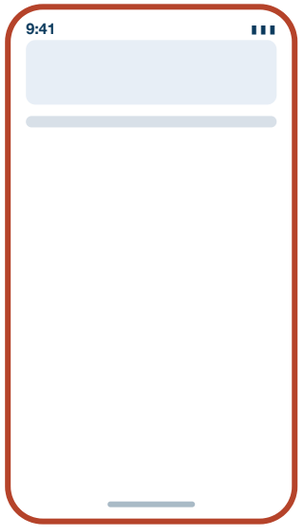
Dos navegadores anidados dibujan uno cada uno. Se elige cuál manda y se apaga el otro — es el caso que vemos en «Anidar».

```
<Stack screenOptions={{ headerShown: false }}>             // a todas
<Stack.Screen name="login" options={{ headerShown: false }} />  // a una, desde el layout
<Stack.Screen options={{ headerShown: false }} />              // a una, desde la pantalla
```

Sin header, lo reponés vos



Con header: empieza donde debe



`headerShown: false` sin nada más

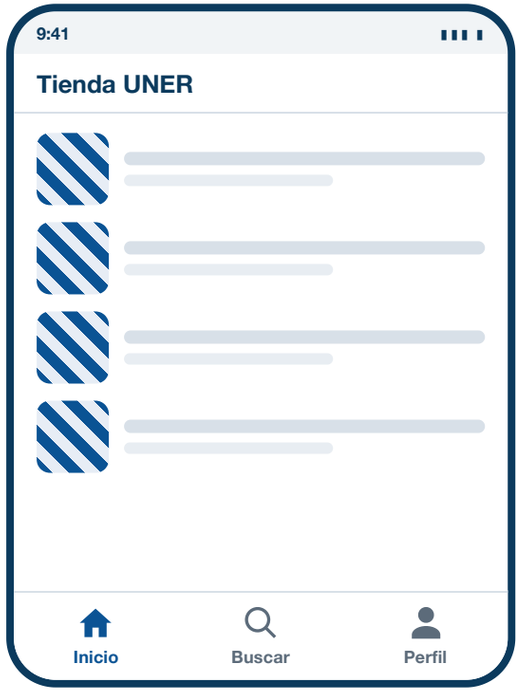
El header no era sólo una barra: te venía resolviendo tres cosas. Al apagarlo, **pasan a ser tuyas**.

- **La safe area de arriba.** Sin header, tu contenido arranca debajo del reloj — es el mismo problema del edge-to-edge.
- **El botón de volver**, con su etiqueta para el lector de pantalla.
- **El título** de la pantalla.

```
const insets = useSafeAreaInsets();

<View style={{ paddingTop: insets.top }}>
  <Pressable onPress={() => router.back()}
             accessibilityLabel="Volver">…
```

Tabs apretá las pestañas



- Secciones **en paralelo**: el usuario salta entre ellas.
- Cada pestaña **recuerda dónde estaba**. Si dejaste un detalle abierto en «Inicio», al volver sigue ahí.
- El ícono y el rótulo salen de `tabBarIcon` y `title`.
- Tres a cinco. Con más, no entran los rótulos.

> Si dos secciones se visitan en secuencia y no en paralelo, no son pestañas: son un Stack.

Tabs: el código

SRC/APP/(TABS)/_LAYOUT.TSX

```tsx
import { Tabs } from 'expo-router';
import Ionicons from '@react-native-vector-icons/ionicons';

export default function Layout() {
  return (
    <Tabs screenOptions={{ tabBarActiveTintColor: '#0B5394' }}>
      <Tabs.Screen name="index" options={{
        title: 'Inicio',
        tabBarIcon: ({ color }) =>
          <Ionicons name="home" size={22} color={color} />,
      }} />
      <Tabs.Screen name="perfil" options={{ title: 'Perfil' }} />
    </Tabs>
  );
}
```

LOS PARÉNTESIS

src/app/

- **_layout.tsx** Stack raíz
- **(tabs)/**
  - **_layout.tsx** las pestañas
  - index.tsx → /
  - perfil.tsx → /perfil

`(tabs)` es un **grupo**: organiza los archivos sin aparecer en la URL. Sin los paréntesis la ruta sería `/tabs/perfil`.

Los íconos vienen de `@react-native-vector-icons`, **un paquete por familia**: `npx expo install @react-native-vector-icons/ionicons`. El `@expo/vector-icons` que van a ver en todos los tutoriales QUEDA DEPRECADO.

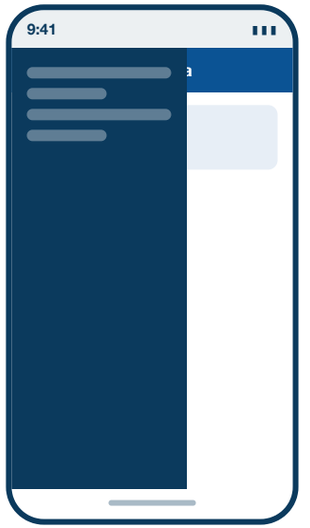
Drawer: el menú lateral

```tsx
// src/app/_layout.tsx
import { GestureHandlerRootView } from 'react-native-gesture-handler';
import { Drawer } from 'expo-router/drawer';

export default function Layout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <Drawer>
        <Drawer.Screen name="index"
          options={{ drawerLabel: 'Inicio', title: 'Tienda' }} />
      </Drawer>
    </GestureHandlerRootView>
  );
}
```

src/app/

- **_layout.tsx** el <Drawer />
- index.tsx → /
- pedidos.tsx → /pedidos



Se abre con el gesto o con el botón ≡

A diferencia de Stack y Tabs, **el Drawer necesita instalación**: `npx expo install @react-navigation/drawer react-native-gesture-handler react-native-reanimated` — y envolver todo en `GestureHandlerRootView`.

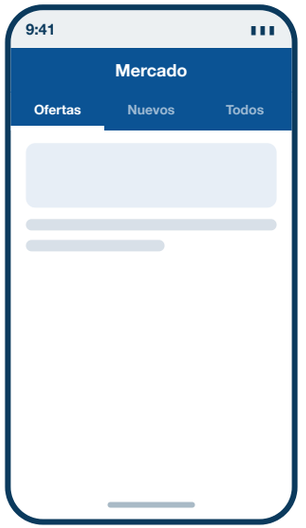
Los que faltan



Modal

No es un navegador aparte: un `Stack.Screen` con `presentation: 'modal'`. Entra desde abajo, la pantalla de atrás queda visible.

Carrito, filtros, un formulario corto.



Pestañas superiores

Las que se pasan deslizando, típicas de Android. `@react-navigation/material-top-tabs`.

Para subdividir una sección, no como navegación principal.

`<Slot />`: el layout que no navega

```tsx
// src/app/_layout.tsx
import { Slot } from 'expo-router';

export default function RootLayout() {
  return (
    <SesionProvider>
      <TemaProvider>
        <Slot />
      </TemaProvider>
    </SesionProvider>
  );
}
```

Un `_layout.tsx` **no recibe** `children`. `<Slot />` es el agujero donde Expo Router dibuja la ruta hija que corresponda.

```
src/app/
├── _layout.tsx  proveedores + <Slot />
└── (tabs)/
    └── _layout.tsx  acá sí navega
```

Cuándo usarlo

- Layout raíz que sólo envuelve **proveedores**: sesión, tema, cliente de datos.
- Cuando hay que **esperar algo** antes de mostrar la app — fuentes, sesión — y recién ahí sacar el splash.
- Cuando la navegación real vive **más abajo**, dentro de un grupo como `(tabs)`.
- Cuando dibujás vos la barra de navegación.

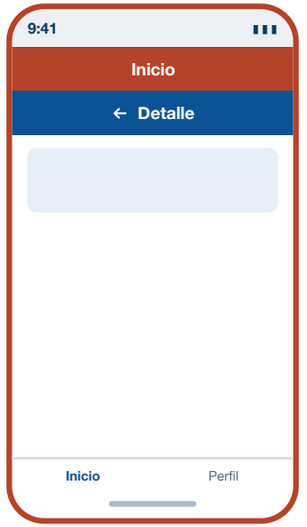
Cuándo no

- Si querés header, transición o gesto de volver → `Stack`.
- Si la pantalla anterior tiene que **seguir viva**: Slot muestra una sola ruta y desmonta la anterior.

Anidar: un Stack por pestaña

```
src/app/
├── _layout.tsx  Stack raíz
└── (tabs)/
    ├── _layout.tsx  Tabs
    ├── inicio/
    │   ├── _layout.tsx  Stack propio
    │   ├── index.tsx  la lista
    │   └── [id].tsx  el detalle
    └── perfil.tsx
```

Es la estructura de casi toda app real: cada sección con su propia historia.



El error clásico: dos headers

> El Stack raíz dibuja uno y el de la pestaña dibuja otro. Se apaga el de arriba: `<Stack.Screen name="(tabs)" options={{ headerShown: false }} />`

UNIDAD 3 · NAVEGACIÓN

Moverse

```
import { Link, router } from 'expo-router';

// Declarativo — preferilo: es accesible y se puede mantener apretado
<Link href="/perfil">Ver perfil</Link>
<Link href={`/producto/${id}`}>Ver producto</Link>
<Link href="/ayuda" asChild>
  <MiBotón>Ayuda</MiBotón>          {/* tu propio componente */}
</Link>

// Imperativo — cuando navegás por una acción
router.push('/perfil');       // apila: queda el botón de volver
router.replace('/inicio');    // reemplaza: sin vuelta atrás
router.back();
router.dismiss();             // cierra un modal
```

> Después de un login va `replace`, no `push`: nadie quiere que «atrás» devuelva al formulario de una sesión ya iniciada.

UNIDAD 3 · NAVEGACIÓN

Parámetros

EN LA RUTA

src/app/producto/

└ **[id].tsx** → /producto/42

```tsx
import { useLocalSearchParams, Stack } from 'expo-router';
import { Text } from 'react-native';

export default function Producto() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const idNum = Number(id);        // '42' → 42

  return (
    <>
      <Stack.Screen options={{ title: `Producto ${id}` }} />
      <Text>Mostrando el producto {idNum}</Text>
    </>
  );
}
```

CÓMO SE LE PASAN

```tsx
import { Link, router, useLocalSearchParams } from 'expo-router';

// en la ruta  →  /producto/42
<Link href={`/producto/${producto.id}`}>Ver</Link>
router.push(`/producto/${producto.id}`);

// como query  →  /buscar?q=zapatillas&orden=precio
<Link href={{
  pathname: '/buscar',
  params: { q: 'zapatillas', orden: 'precio' },
}}>Buscar</Link>

// y del otro lado, en src/app/buscar.tsx
const { q, orden } = useLocalSearchParams<{
  q?: string; orden?: string;
}>();
```

Todo parámetro llega como texto. Ese `id` es `'42'`, no `42`. Comparado con `===` contra un número, no coincide nunca — y el código parece correcto.

Rutas protegidas

src/app/

- **_layout.tsx** decide qué rutas existen
- login.tsx → /login
- **(privado)/** sólo con sesión
  - perfil.tsx → /perfil

```tsx
export default function Layout() {
  const { sesion } = useSesion();

  return (
    <Stack>
      <Stack.Protected guard={!!sesion}>
        <Stack.Screen name="(privado)" />
      </Stack.Protected>

      <Stack.Protected guard={!sesion}>
        <Stack.Screen name="login" />
      </Stack.Protected>
    </Stack>
  );
}
```

> Si el `guard` es falso, **la ruta no existe**: no se llega ni escribiendo la URL. Mejor que redirigir desde adentro, que alcanza a mostrar la pantalla privada un instante.

Documentación

- **Expo Router**
  `docs.expo.dev/router/introduction`
- **React Navigation** — las opciones avanzadas de navegadores y headers se documentan acá
  `reactnavigation.org/docs/getting-started`
- **Íconos** — instalación, tipografía de íconos propia y el componente botón
  `docs.expo.dev/guides/icons`
  el catálogo para buscar el nombre de cada ícono: `oblador.github.io/react-native-vector-icons`